Amauri Hopewell

Southern New Hampshire University

CS-499 Computer Science Capstone

October 4, 2026

Milestone Four: Databases Enhancement

1. What is the artifact, and when was it created?

This is the Grazioso Salvare / Austin Animal Center work from CS-340 Client/Server Development, 2025 C-5 (September–October 2025). The original files are in the original/ folder of this zip and at https://github.com/AmauriHopewell/CS340. CRUD_Python_Module_5AH.py is the AnimalShelter class allowing create, read, update, and delete operations against MongoDB database aac, collection animals. ProjectTwoDashboard.ipynb is a Dash notebook that filters those rows by breed, age, and sex so staff can find training candidates. Read used an aggregation pipeline that grouped on animal_id to hide duplicates. The constructor took a username and password, with defaults still in the source file. This milestone enhances that CRUD module and dashboard.

2. Why this artifact, and how was it improved?

I picked this project because the original assignment was talking to MongoDB correctly and then using those queries from a client. It already had real CRUD and a real collection. However, it had some database gaps: credentials were in the constructor, a caller dict passed straight into $match, uniqueness was handled in the query instead of in the schema, there were no indexes on the fields the dashboard filters, and the app was run from a Jupyter notebook.

I kept the AnimalShelter class in enhanced/animal_shelter.py, and kept using MongoDB, but strengthened how it was used, by making MONGO_USER and MONGO_PASS come from

the environment. If they are missing, the constructor raises ConfigurationError and does not open a client. To add security, I removed default credentials in this file, and created role-based access to Staff may read and filter, Admin may create, update, and delete, and both roles use the Mongo login from the environment.

The role argument is what limits writes. sanitize_lookup allow-lists field names and a short list of operators ($gte, $lte, $in, $eq). A $where key is rejected, and read uses find. Also, I created a unique index on animal_id to stop duplicates at write time. I placed extra indexes on breed, age_upon_outcome_in_weeks, and sex_upon_outcome. counts_by_breed_and_outcome is an aggregation that counts by breed and outcome_type. The Module One sketch grouped on rescue_type, and the AAC collection groups on outcome_type, so that is the field I grouped on. dashboard.py is the same Grazioso filters as a normal Python process. I have set many to default to false, so update and delete touch one matching document unless the caller passes many=True. An empty lookup on those writes returns 0, while create(None) raises QueryError. Additionally, python -m pytest tests -q is 10 tests on mongomock, so the zip does not need a live MongoDB on my laptop.

The trade-off is the unique index. If the AAC dump already has duplicate animal_id values, ensure_indexes fails until those rows are cleaned. A $group on read would hide those rows, but a unique index rejects them on write. However, dashboard.py still opens if that call fails, so a dirty import does not take down the filter UI. That is the trade-off I mentioned in the Module One plan.

3. Course outcomes from Module One

The Module One plan said this enhancement would show outcome 5 first, then 3 and 4. Outcome 5 is taking the secrets (like credentials) out of the source code, creating an allow-list on queries, and differentiating between staff versus admin. Outcome 4 is using PyMongo, environment config,

and indexes together. Outcome 3 is comparing unique index versus aggregation-for-deduplication. Outcome 2 satisfied by this write-up. For Outcome 1, I will create a GitHub branch after I take comments.

4. What I learned, and what got in the way

The first major problem with this code was the constructor defaults sitting in Git. To fix it, I used a fail-closed with credentials from the environment. The connect log records host, port, database, and username, but not password. Tests check that AnimalShelter(admin) with MONGO_USER and MONGO_PASS unset raises ConfigurationError. The original read method checked that lookup was a dict. A dict can still hold $where. To improve security, sanitize_lookup now protects field names and operators from malicious input. Staff calling now delete raises PermissionDenied. Empty lookup on update or delete returns 0, so a blank filter does not touch every document. Also, create still takes a dict and inserts it. Duplicate animal_id is what the unique index catches on write. The original notebook had a map, and dashboard.py has the Grazioso breed, age, and sex filters, a table, and a pie chart. I left the notebook in original/ and skipped a new JavaScript front end, which I will implement later now that the data layer is done.

References

OWASP Foundation. (n.d.). OWASP code review guide. https://owasp.github.io/www-project-code-review-guide/

Acknowledgment of generative AI use

I used Grok 4.6 (xAI) to help organize this narrative against the Milestone Four prompts, to format the Word file, and to improve the writing.